Introduction to Claude Code

Weekend 1, Saturday parallel session

CAS Building ML/AI Applications, ETH Zurich

After the weekend 1 lecture of From Data to Decisions, by Francesco Caperna and Deli

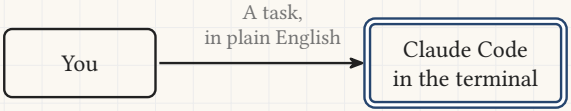
Saturday 5 September 2026

1

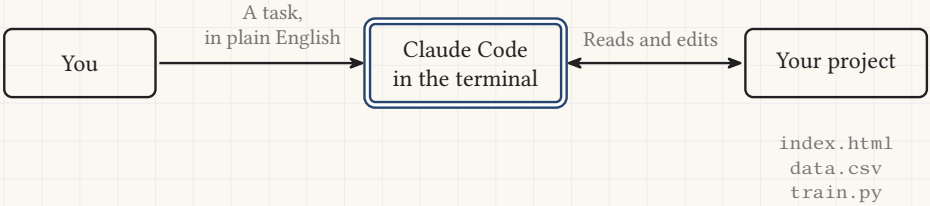
Section 1

Claude Code

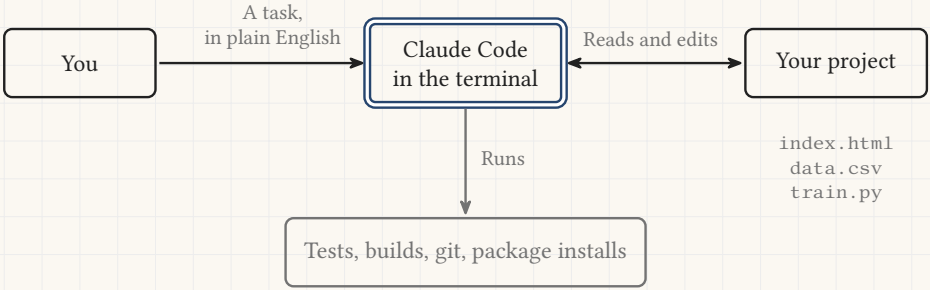
What Claude Code is



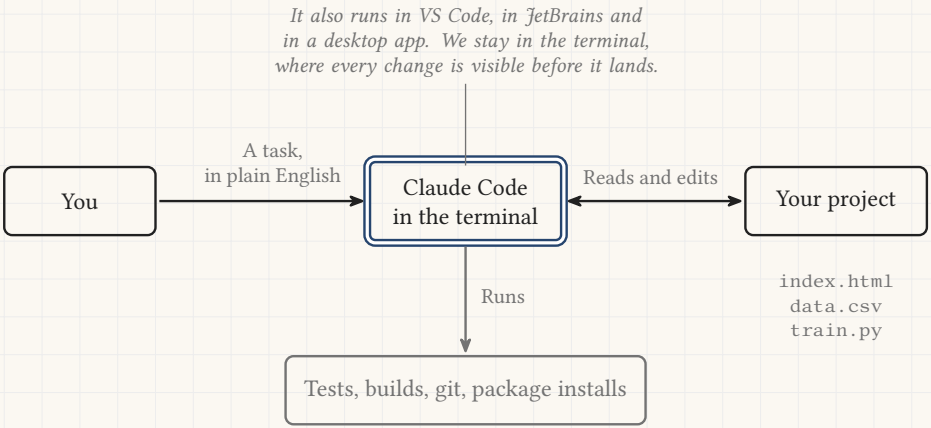
What Claude Code is



What Claude Code is



What Claude Code is



Why this course teaches it

It does not solve every problem. It lets you try more solutions in the same hour.

Why this course teaches it

It does not solve every problem. It lets you try more solutions in the same hour.

The coding exercises of every weekend

A stub to fill in reads faster with a reader beside you

Why this course teaches it

It does not solve every problem. It lets you try more solutions in the same hour.

The coding exercises of every weekend

A stub to fill in reads faster with a reader beside you

The weekend project, built end to end

A full pipeline is more code than a weekend has hours

Why this course teaches it

It does not solve every problem. It lets you try more solutions in the same hour.

The coding exercises of every weekend

A stub to fill in reads faster with a reader beside you

The weekend project, built end to end

A full pipeline is more code than a weekend has hours

Your own work, after the course

The licence outlives the weekend it was bought for

What you need on your laptop

Three things. Two of them you install once, and one is already there.

VS Code

A free text editor from Microsoft. You install it

What you need on your laptop

Three things. Two of them you install once, and one is already there.

VS Code

A free text editor from Microsoft. You install it

A terminal

A window where you type commands. It is already on your machine

What you need on your laptop

Three things. Two of them you install once, and one is already there.

VS Code

A free text editor from Microsoft. You install it

A terminal

A window where you type commands. It is already on your machine

Claude Code

The program itself. It runs inside the terminal

What you need on your laptop

Three things. Two of them you install once, and one is already there.

VS Code

A free text editor from Microsoft. You install it

A terminal

A window where you type commands. It is already on your machine

Claude Code

The program itself. It runs inside the terminal

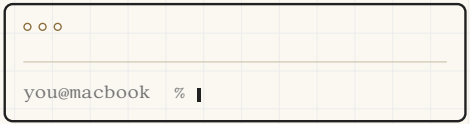
Every step is written out in the setup guide, with a button that copies each command, so nothing has to be typed by hand.

`eth-bmai-hs26.github.io/BMAI-PAGE/guides/claude-code-setup.html`

Opening a terminal

On a Mac

- Press `Cmd` + `Space`.
- Type `Terminal`, press `Enter`.

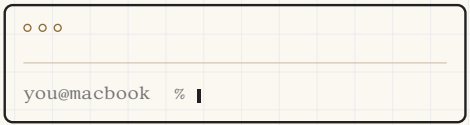


On Windows

Opening a terminal

On a Mac

- Press `Cmd` + `Space`.
- Type `Terminal`, press `Enter`.



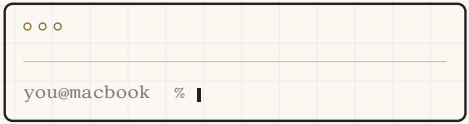
On Windows

- Press `Win` + `X`.
- Choose **Windows PowerShell**, or **Terminal**.

Opening a terminal

On a Mac

- Press `Cmd` + `Space`.
- Type `Terminal`, press `Enter`.



On Windows

- Press `Win` + `X`.
- Choose **Windows PowerShell**, or **Terminal**.

- You type one line and press `Enter`. Nothing happens until you do.
- You cannot click on anything in there. Use the arrow keys.
- `Esc` interrupts Claude. `exit` closes it.

Installing it, and signing in

```
macOS, Linux, WSL
$ curl -fsSL https://claude.ai/install.sh | bash

Windows PowerShell
$ irm https://claude.ai/install.ps1 | iex

$ claude --version
2.1.211 (Claude Code)
```

Installing it, and signing in

```
macOS, Linux, WSL
$ curl -fsSL https://claude.ai/install.sh | bash

Windows PowerShell
$ irm https://claude.ai/install.ps1 | iex

$ claude --version
2.1.211 (Claude Code)
```

- **Do not type these.** Open the setup guide and press the copy button beside each one.

Installing it, and signing in

```
macOS, Linux, WSL
$ curl -fsSL https://claude.ai/install.sh | bash

Windows PowerShell
$ irm https://claude.ai/install.ps1 | iex

$ claude --version
2.1.211 (Claude Code)
```

- **Do not type these.** Open the setup guide and press the copy button beside each one.
- You were sent an **email inviting you to join the course's Claude team**. Accept it, then sign in with that account the first time you run `claude`.

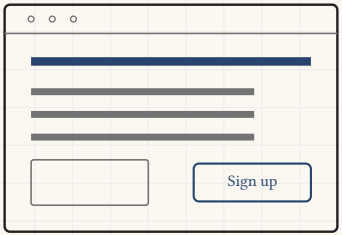
2

Section 2

Git, the safety net

Why Git

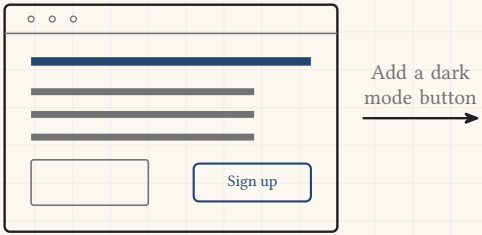
You ask for a small change. Claude edits your files, and the page comes back different.



Before: everything works

Why Git

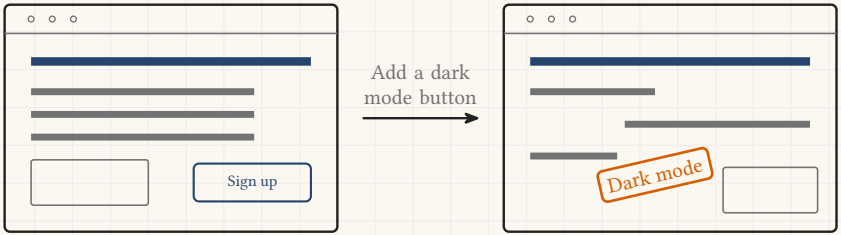
You ask for a small change. Claude edits your files, and the page comes back different.



Before: everything works

Why Git

You ask for a small change. Claude edits your files, and the page comes back different.

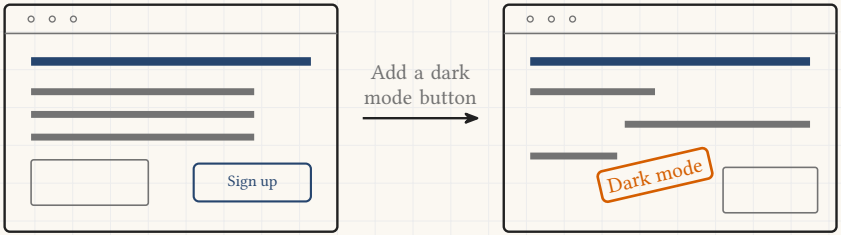


Before: everything works

After: the layout falls apart

Why Git

You ask for a small change. Claude edits your files, and the page comes back different.

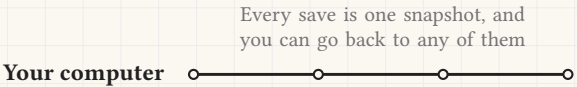


How Git works

Git keeps a snapshot of the project every time you save one, and nothing is ever written over.

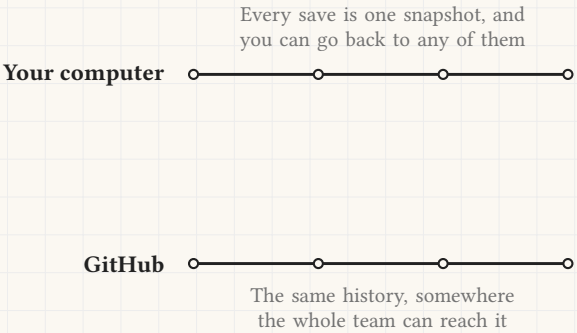
How Git works

Git keeps a snapshot of the project every time you save one, and nothing is ever written over.



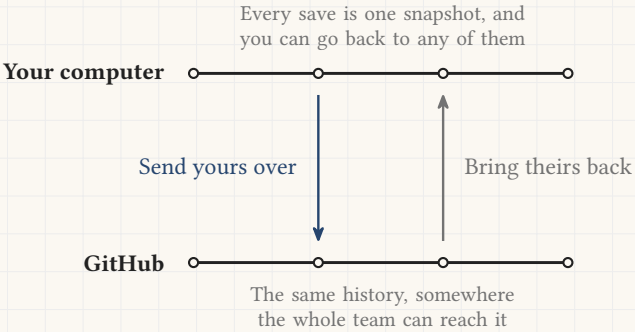
How Git works

Git keeps a snapshot of the project every time you save one, and nothing is ever written over.



How Git works

Git keeps a snapshot of the project every time you save one, and nothing is ever written over.



You never type any of this

Four words name the four moves. You say them in English, Claude runs them, and it reports every command it ran.

- `add`, choose what goes into the snapshot
- `commit`, take the snapshot
- `push`, send it to GitHub
- `pull`, bring back what others sent

You never type any of this

Four words name the four moves. You say them in English, Claude runs them, and it reports every command it ran.

- `add`, choose what goes into the snapshot
- `commit`, take the snapshot
- `push`, send it to GitHub
- `pull`, bring back what others sent

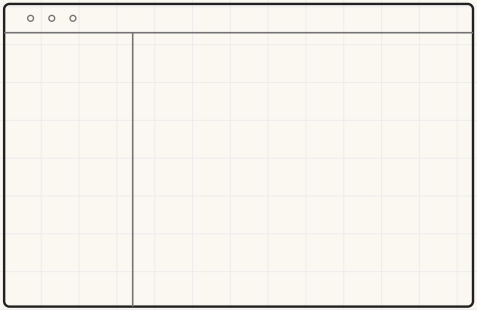
```
> Save what we have and put it on GitHub.
Running git add .
Running git commit -m "feat: add hero"
Running git push
Done. 1 file changed.
```

3

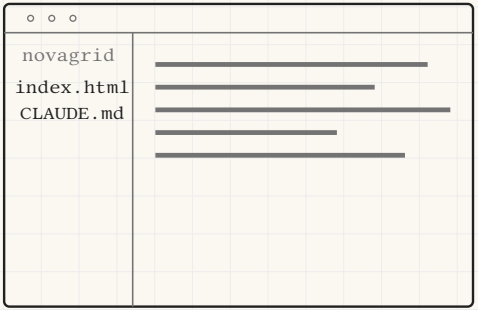
Section 3

A session from scratch

Step 1: the project folder and its terminal

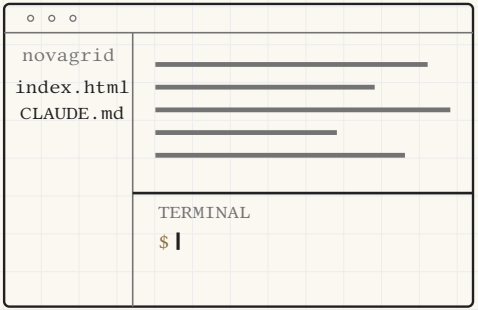


Step 1: the project folder and its terminal



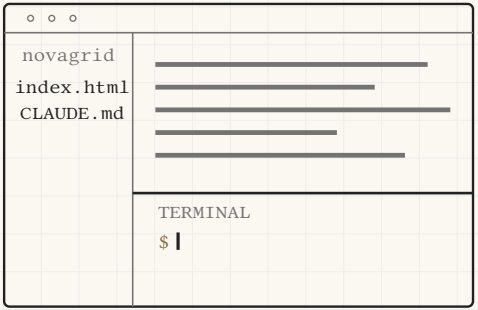
- Open the folder in VS Code: **File**, then **Open Folder**.

Step 1: the project folder and its terminal



- Open the folder in VS Code: **File**, then **Open Folder**.
- Open the built in terminal: **Terminal**, then **New Terminal**.
- Or press `Ctrl` + `` ` ``.

Step 1: the project folder and its terminal



- Open the folder in VS Code: **File**, then **Open Folder**.
- Open the built in terminal: **Terminal**, then **New Terminal**.
- Or press `Ctrl` + `` ` ``.
- Every command in this lecture is typed in that panel.

Step 2: turn the folder into a repository

```
$ git init
Initialized empty Git repository in
/Users/you/novagrid/.git/
```

`git init` creates the hidden `.git` folder. The project is now tracked.

Step 2: turn the folder into a repository

```
$ git init
Initialized empty Git repository in
/Users/you/novagrid/.git/
```

`git init` creates the hidden `.git` folder. The project is now tracked.

```
$ git add .
$ git commit -m "Initial commit"
[main (root-commit) a1b2c3d] Initial commit
```

`git add` chooses what to save. `git commit` saves it as a snapshot you can always return to.

Step 3: start Claude Code inside the repository

```
$ claude
Welcome to Claude Code!
>
```

Step 3: start Claude Code inside the repository

```
$ claude
Welcome to Claude Code!
>
```

- The prompt is now Claude's. Type in plain English.

Step 3: start Claude Code inside the repository

```
$ claude
Welcome to Claude Code!
>
```

- The prompt is now Claude's. Type in plain English.
- It starts in the folder you are standing in, so it sees that project and no other.

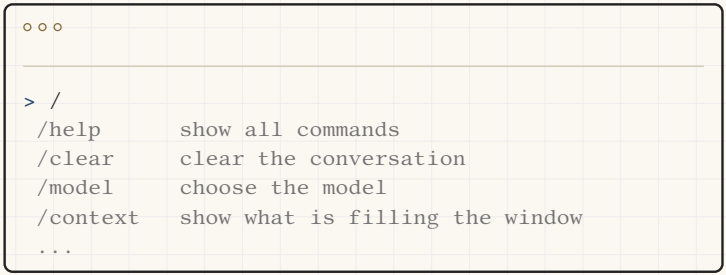
Step 3: start Claude Code inside the repository

```
$ claude
Welcome to Claude Code!
>
```

- The prompt is now Claude's. Type in plain English.
- It starts in the folder you are standing in, so it sees that project and no other.
- Because the folder is a repository, every change it makes is one `git diff` away from being reviewed or undone.

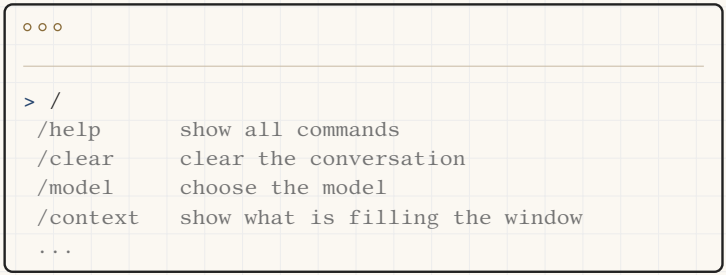
The command menu

Type a forward slash to open the command menu. Keep typing to filter it, then press enter.



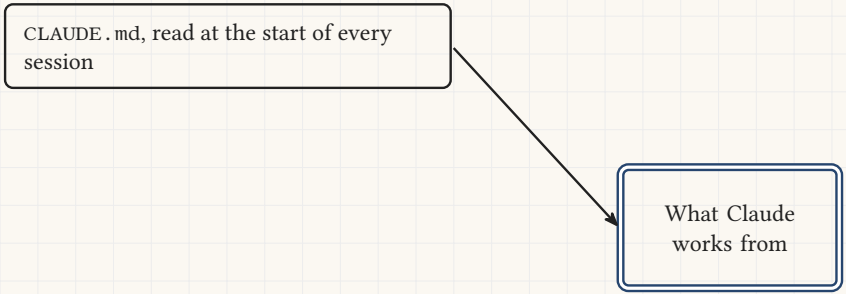
The command menu

Type a forward slash to open the command menu. Keep typing to filter it, then press enter.

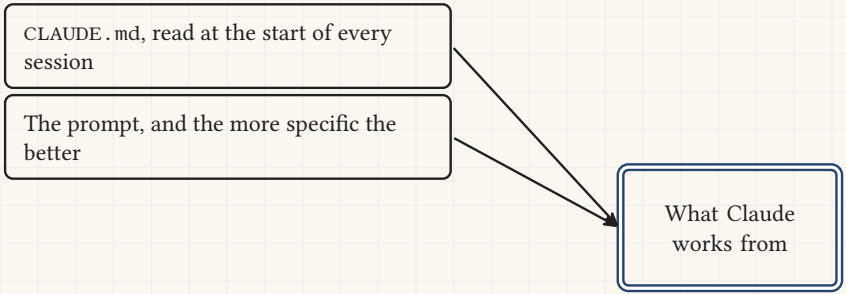


Everything Claude Code can be told to do about itself lives behind that slash.

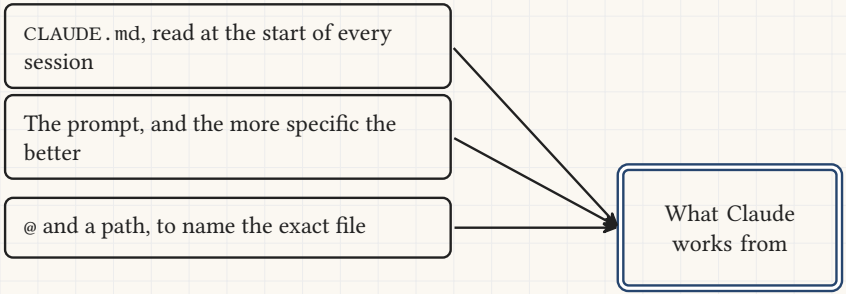
Five ways to give Claude context



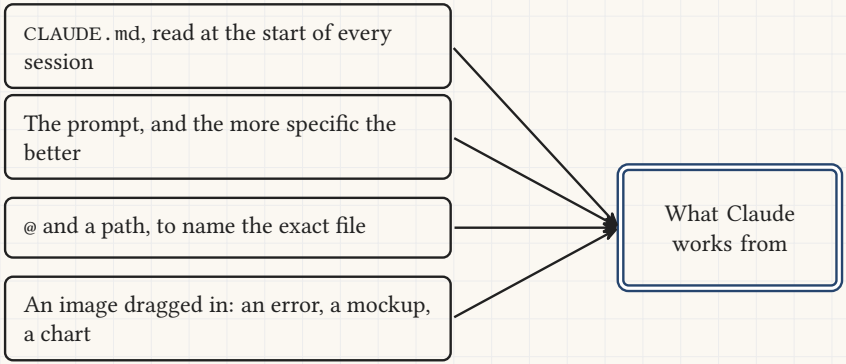
Five ways to give Claude context



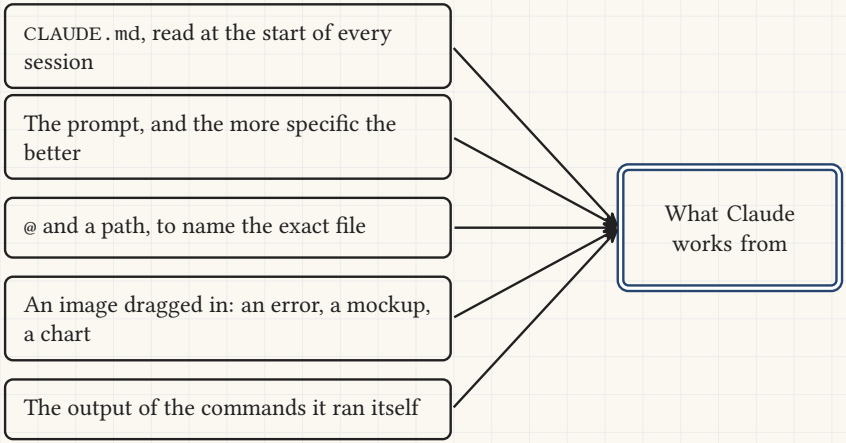
Five ways to give Claude context



Five ways to give Claude context



Five ways to give Claude context



CLAUDE.md, the file it reads first

CLAUDE.md

```
## Commands
- Run tests: pytest
- Start app: python main.py

## Project structure
- src/   core code
- data/  raw datasets, do not edit

## Conventions

- Use type hints everywhere
```

CLAUDE.md, the file it reads first

CLAUDE.md

```
## Commands
- Run tests: pytest
- Start app: python main.py

## Project structure
- src/   core code
- data/  raw datasets, do not edit

## Conventions

- Use type hints everywhere
```

- How to build, test and run the project.
- Where things are, and which files matter.
- The conventions, and the things to avoid.

CLAUDE.md, the file it reads first

CLAUDE.md

```
## Commands
- Run tests: pytest
- Start app: python main.py

## Project structure
- src/   core code
- data/  raw datasets, do not edit

## Conventions

- Use type hints everywhere
```

- How to build, test and run the project.
- Where things are, and which files matter.
- The conventions, and the things to avoid.

```
> /init
Analyzing your project...
Created CLAUDE.md
```

CLAUDE.md, the file it reads first

CLAUDE.md

```
## Commands
- Run tests: pytest
- Start app: python main.py

## Project structure
- src/   core code
- data/  raw datasets, do not edit

## Conventions

- Use type hints everywhere
```

- How to build, test and run the project.
- Where things are, and which files matter.
- The conventions, and the things to avoid.

```
ooo

> /init
Analyzing your project...
Created CLAUDE.md
```

It writes a first version. Correct it from there.

Pointing at a file, and making it remember

A path after @ pulls that file in.

Pointing at a file, and making it remember

```
○ ○ ○

> Explain what @src/main.py does
```

A path after `@` pulls that file in.

```
○ ○ ○

> Remember that we always use pytest
Saved to memory.
```

Ask it to remember, and the note survives the session.

Pointing at a file, and making it remember

```
> Explain what @src/main.py does
```

A path after @ pulls that file in.

```
> Remember that we always use pytest
Saved to memory.
```

Ask it to remember, and the note survives the session.

```
> /memory
Lists your memory files.
Select one to open it.
```

What it remembers is a plain text file you can edit.

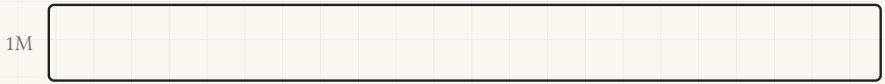
4

Section 4

Getting more out of it

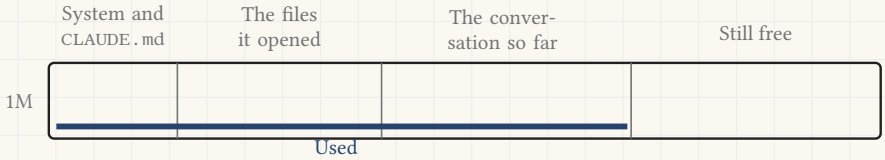
The context window

Your code, the files it opened and the whole conversation share one window, measured in tokens.



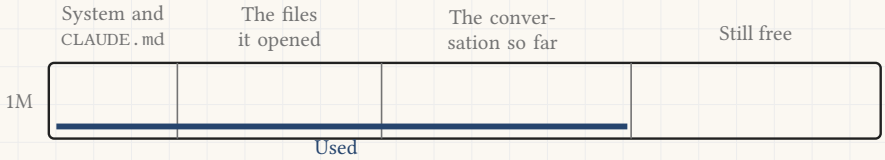
The context window

Your code, the files it opened and the whole conversation share one window, measured in tokens.



The context window

Your code, the files it opened and the whole conversation share one window, measured in tokens.




```
> /context
Context: 700k / 1M tokens used (70%)
```

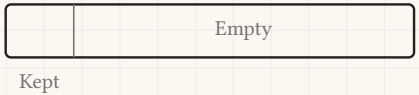
Clearing and compacting

```
> /clear
Context cleared. Starting fresh.
```

Everything goes except `CLAUDE.md`, which is read again. Use it when the next task is unrelated.

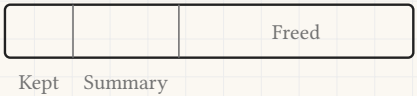
Clearing and compacting

```
> /clear
Context cleared. Starting fresh.
```



Everything goes except `CLAUDE.md`, which is read again. Use it when the next task is unrelated.

```
> /compact
Summarizing conversation...
Context compacted, tokens freed.
```



The conversation becomes a summary of itself. Use it when the next task continues this one.

Choosing the model

Fable, for work larger than one sitting

Choosing the model

Fable, for work larger than one sitting

Opus, for the complex and the tricky

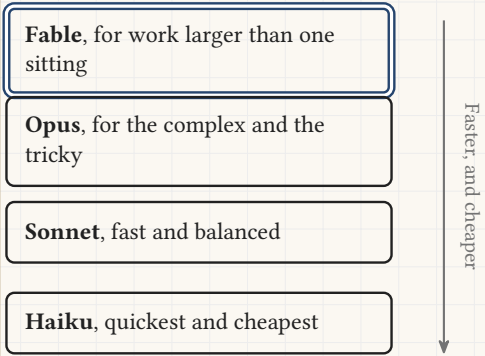
Choosing the model

Fable, for work larger than one sitting

Opus, for the complex and the tricky

Sonnet, fast and balanced

Choosing the model



Choosing the model

Fable, for work larger than one sitting

Opus, for the complex and the tricky

Sonnet, fast and balanced

Haiku, quickest and cheapest

Faster, and cheaper

```
> /model
Fable     most capable, chosen
Opus      complex and tricky
Sonnet    fast and balanced
Haiku     quickest
```

Choosing the model

Fable, for work larger than one sitting

Opus, for the complex and the tricky

Sonnet, fast and balanced

Haiku, quickest and cheapest

Faster, and cheaper

```
> /model
Fable     most capable, chosen
Opus      complex and tricky
Sonnet    fast and balanced
Haiku     quickest
```

Match the model to the task. A footer does not need the most capable one. A subtle bug does.

Three ways to work

Edit

It reads and edits your files to carry the task out. Best when the task is clear.

The default

Three ways to work

Edit

It reads and edits your files to carry the task out. Best when the task is clear.

The default

Plan

It researches and proposes the steps, and changes nothing until you approve.

`/plan`, or Shift + Tab

Three ways to work

Edit

It reads and edits your files to carry the task out. Best when the task is clear.

The default

Plan

It researches and proposes the steps, and changes nothing until you approve.

`/plan`, or Shift + Tab

Effort

It reasons longer before acting. For a tricky bug or a design decision.

`/effort`

Three ways to work

Edit

It reads and edits your files to carry the task out. Best when the task is clear.

The default

Plan

It researches and proposes the steps, and changes nothing until you approve.

`/plan`, or Shift + Tab

Effort

It reasons longer before acting. For a tricky bug or a design decision.

`/effort`

Shift + Tab cycles how much Claude may do without asking: every edit confirmed, edits accepted automatically, or plan mode.

Writing your own command

A Markdown file in `.claude/commands/` becomes a command you can call by its own name.

`.claude/commands/qa-check.md`

```
Check index.html for leftover
placeholder text, dead links and
missing alt attributes. Report
each one with a severity.
```

Writing your own command

A Markdown file in `.claude/commands/` becomes a command you can call by its own name.

`.claude/commands/qa-check.md`

```
Check index.html for leftover
placeholder text, dead links and
missing alt attributes. Report
each one with a severity.
```



```
> /qa-check
Reading index.html...
3 issues found.
```

Writing your own command

A Markdown file in `.claude/commands/` becomes a command you can call by its own name.

`.claude/commands/qa-check.md`

```
Check index.html for leftover
placeholder text, dead links and
missing alt attributes. Report
each one with a severity.
```

```
ooo

> /qa-check
Reading index.html...
3 issues found.
```

Good for a check you run every time, written down once so nobody has to remember it.

Commands worth knowing

Knowing that the features exist is what matters. Ask in plain English for the rest.

- `/help`, list every command
- `/init`, write a first `CLAUDE.md`
- `/memory`, read and edit what it remembers
- `/context`, show what fills the window
- `/clear`, start the conversation again
- `/compact`, summarize it to free space
- `/model` and `/effort`, pick the engine
- `/review`, review a diff or a pull request

Commands worth knowing

Knowing that the features exist is what matters. Ask in plain English for the rest.

- `/help`, list every command
- `/init`, write a first `CLAUDE.md`
- `/memory`, read and edit what it remembers
- `/context`, show what fills the window
- `/clear`, start the conversation again
- `/compact`, summarize it to free space
- `/model` and `/effort`, pick the engine
- `/review`, review a diff or a pull request

```
ooo

> How do I free up the context?
Use /compact to summarize the conversation.
```

Two things to know exist*

Subagents

- For a big task, Claude can hand parts of the work to helpers that run on their own.
- Each one carries its own context, so the main conversation stays small.
- Manage them with `/agents`.

MCP

Two things to know exist*

Subagents

- For a big task, Claude can hand parts of the work to helpers that run on their own.
- Each one carries its own context, so the main conversation stays small.
- Manage them with `/agents`.

MCP

- The Model Context Protocol connects Claude Code to tools outside your folder.
- Databases, GitHub, Slack, a browser, and so on.
- Add a server with `claude mcp add`, inspect them with `/mcp`.

Two things to know exist*

Subagents

- For a big task, Claude can hand parts of the work to helpers that run on their own.
- Each one carries its own context, so the main conversation stays small.
- Manage them with `/agents`.

Neither is needed for this morning's exercise.

MCP

- The Model Context Protocol connects Claude Code to tools outside your folder.
- Databases, GitHub, Slack, a browser, and so on.
- Add a server with `claude mcp add`, inspect them with `/mcp`.

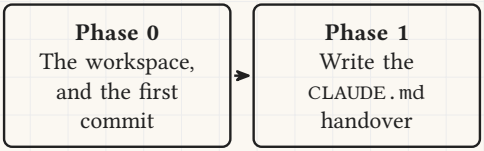
The exercise: Operation Midnight Launch

It is 11 at night. The freelancer who was building the investor demo page has vanished, the investors arrive at nine, and you have one engineer left.

Phase 0
The workspace, and the first commit

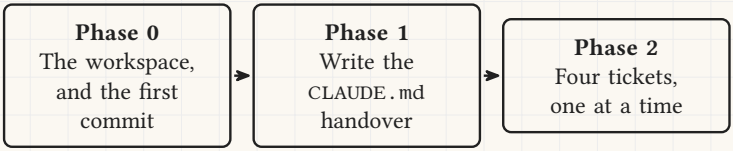
The exercise: Operation Midnight Launch

It is 11 at night. The freelancer who was building the investor demo page has vanished, the investors arrive at nine, and you have one engineer left.



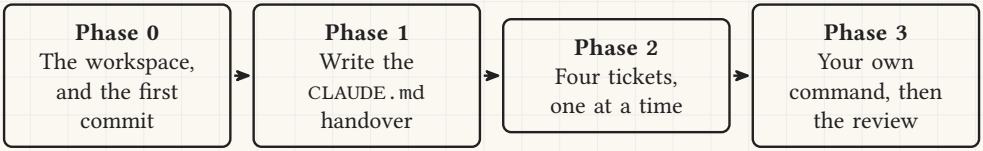
The exercise: Operation Midnight Launch

It is 11 at night. The freelancer who was building the investor demo page has vanished, the investors arrive at nine, and you have one engineer left.



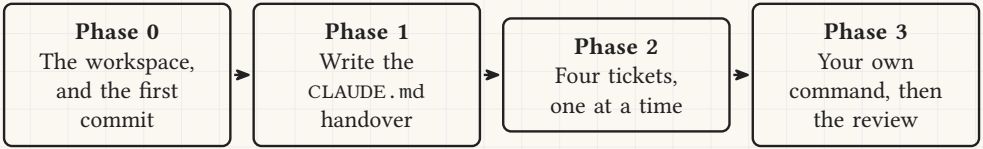
The exercise: Operation Midnight Launch

It is 11 at night. The freelancer who was building the investor demo page has vanished, the investors arrive at nine, and you have one engineer left.



The exercise: Operation Midnight Launch

It is 11 at night. The freelancer who was building the investor demo page has vanished, the investors arrive at nine, and you have one engineer left.



The notebook, and how to open it, are both in the setup guide.

`eth-bmai-hs26.github.io/BMAI-PAGE/exercises/we1/operation-midnight-launch.ipynb`

Where to read more

- This course's setup guide, with a copy button on every command:
  `eth-bmai-hs26.github.io/BMAI-PAGE/guides/claude-code-setup.html`
- The first session, step by step:
  `code.claude.com/docs/en/quickstart`

Where to read more

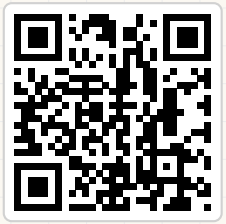
- This course's setup guide, with a copy button on every command:
  `eth-bmai-hs26.github.io/BMAI-PAGE/guides/claude-code-setup.html`
- The first session, step by step:
  `code.claude.com/docs/en/quickstart`
- Every command:
  `code.claude.com/docs/en/commands`

Where to read more

- This course's setup guide, with a copy button on every command:
  `eth-bmai-hs26.github.io/BMAI-PAGE/guides/claude-code-setup.html`
- The first session, step by step:
  `code.claude.com/docs/en/quickstart`
- Every command:
  `code.claude.com/docs/en/commands`
- How it remembers a project:
  `code.claude.com/docs/en/memory`

Where to read more

- This course's setup guide, with a copy button on every command:
  `eth-bmai-hs26.github.io/BMAI-PAGE/guides/claude-code-setup.html`
- The first session, step by step:
  `code.claude.com/docs/en/quickstart`
- Every command:
  `code.claude.com/docs/en/commands`
- How it remembers a project:
  `code.claude.com/docs/en/memory`



`code.claude.com/docs`